

Module : Optimisation

Apprentissage Fédéré pour la Détection de Maladies Cardiaques

*Simulation d'un système d'apprentissage collaboratif
préservant la confidentialité des données médicales*

Option 4 : Simulation d'apprentissage fédéré sur données médicales

Réalisé par :

Sifeddine LEGNIOUI
Youssef SARRAF

Encadré par :

Pr. [NOM DE L'ENCADRANT]

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet. Ce travail, réalisé dans le cadre du module d’Optimisation, a constitué une opportunité exceptionnelle d’approfondir nos connaissances théoriques et pratiques dans le domaine de l’apprentissage automatique distribué, et plus particulièrement dans le paradigme novateur de l’apprentissage fédéré appliqué au secteur de la santé.

Nos remerciements les plus sincères s’adressent en premier lieu à notre encadrant, dont les orientations pédagogiques, les conseils méthodologiques et la disponibilité constante ont été déterminants pour la conduite de ce projet. Ses explications sur les fondements mathématiques de l’optimisation distribuée et ses remarques constructives lors des séances de suivi nous ont permis de structurer notre démarche scientifique de manière rigoureuse et de maintenir un cap clair tout au long de notre travail.

Nous souhaitons également remercier l’ensemble du corps professoral du département d’Informatique, dont les enseignements fondamentaux en mathématiques, en statistiques, en apprentissage automatique et en programmation ont constitué le socle indispensable sur lequel repose ce projet. Les compétences acquises au cours de notre formation en Ingénierie des Données et Intelligence Artificielle se sont révélées essentielles pour la compréhension des algorithmes d’optimisation fédérée et pour leur implémentation correcte.

Nous exprimons notre reconnaissance envers la communauté open-source, et en particulier les développeurs des bibliothèques PyTorch, scikit-learn, NumPy et Matplotlib, dont les outils ont permis de concrétiser nos idées de manière efficace. La disponibilité du dataset Heart Disease sur le dépôt UCI Machine Learning Repository a également été un facteur déterminant dans le choix de notre sujet, et nous remercions les chercheurs à l’origine de sa collecte et de sa mise à disposition publique.

Enfin, nous adressons nos remerciements les plus chaleureux à nos familles et à nos proches, dont le soutien moral et les encouragements constants ont été une source de motivation inépuisable. Leur patience et leur compréhension durant les longues heures de travail consacrées à ce projet méritent une reconnaissance toute particulière. Ce travail est le fruit d’un effort collaboratif soutenu, et chaque contribution, même indirecte, a participé à son aboutissement.

Résumé

Résumé — L'apprentissage fédéré (Federated Learning) constitue un paradigme d'apprentissage automatique distribué qui permet à plusieurs entités de collaborer à l'entraînement d'un modèle partagé sans jamais échanger leurs données brutes. Ce rapport présente une étude comparative approfondie de quatre algorithmes d'optimisation fédérée — Local SGD, FedAvg, FedAdam et FedProx — dans le contexte de la détection de maladies cardiaques. Nous simulons une fédération de huit hôpitaux exploitant le dataset Heart Disease UCI (1025 patients, 13 attributs cliniques) à travers un perceptron multicouche (MLP) de 2 657 paramètres. Deux scénarios de distribution des données sont évalués : une partition IID (équilibrée) et une partition Non-IID (Dirichlet $\alpha = 0.5$). Les résultats expérimentaux démontrent que FedAvg atteint une accuracy de 91,3% en Non-IID, tandis que FedProx offre la convergence la plus stable grâce à son terme de régularisation proximale ($\mu = 0.1$). L'analyse de la divergence L_2 entre modèles locaux confirme l'efficacité du terme proximal pour réduire la dérive des clients. Le coût de communication total est estimé à environ 2,49 Mo pour 15 rounds, démontrant la faisabilité d'un déploiement en environnement hospitalier réel.

Mots-clés : apprentissage fédéré, optimisation distribuée, maladies cardiaques, FedAvg, FedProx, FedAdam, hétérogénéité statistique, confidentialité des données médicales.

Abstract — Federated Learning (FL) is a distributed machine learning paradigm that enables multiple entities to collaboratively train a shared model without ever exchanging their raw data. This report presents a comprehensive comparative study of four federated optimization algorithms — Local SGD, FedAvg, FedAdam, and FedProx — in the context of heart disease detection. We simulate a federation of eight hospitals using the UCI Heart Disease dataset (1025 patients, 13 clinical attributes) with a multilayer perceptron (MLP) of 2,657 parameters. Two data distribution scenarios are evaluated : an IID (balanced) partition and a Non-IID (Dirichlet $\alpha = 0.5$) partition. Experimental results show that FedAvg achieves an accuracy of 91.3% under Non-IID conditions, while FedProx provides the most stable convergence thanks to its proximal regularization term ($\mu = 0.1$). The L_2 divergence analysis between local models confirms the effectiveness of the proximal term in reducing client drift. The total communication cost is estimated at approximately 2.49 MB over 15 rounds, demonstrating the feasibility of deployment in real hospital environments.

Keywords : federated learning, distributed optimization, heart disease, FedAvg, Fed-Prox, FedAdam, statistical heterogeneity, medical data privacy.

Table des matières

Remerciements	1
Résumé / Abstract	2
1 Introduction Générale	9
1.1 Contexte général	9
1.2 Problématique	9
1.3 L'Apprentissage Fédéré comme solution	10
1.4 Objectifs du projet	11
1.5 Organisation du rapport	11
2 État de l'Art et Fondements Théoriques	12
2.1 Formalisme de l'optimisation en apprentissage automatique	12
2.2 De l'optimisation centralisée à l'optimisation distribuée	12
2.3 L'apprentissage fédéré : architecture et protocole	13
2.4 Le problème de l'hétérogénéité statistique (Non-IID)	13
2.5 Algorithme FedAvg	14
2.6 Algorithme FedAdam	15
2.7 Algorithme FedProx	16
2.8 Local SGD et apprentissage isolé	17
2.9 Synthèse comparative théorique	17
3 Méthodologie et Implémentation	18
3.1 Choix du dataset et justification	18
3.2 Pipeline de préparation des données	19
3.3 Simulation de la fédération hospitalière	19
3.3.1 Partitionnement IID	19
3.3.2 Partitionnement Non-IID par distribution de Dirichlet	19
3.4 Architecture du modèle neuronal	20
3.5 Infrastructure logicielle et boucle fédérée	20
3.6 Hyperparamètres	21
3.7 Métriques d'évaluation implémentées	22

3.8	Environnement technique	23
4	Résultats Expérimentaux et Analyse	24
4.1	Distribution des données après partitionnement	24
4.1.1	Analyse du scénario IID	24
4.1.2	Analyse du scénario Non-IID	24
4.2	Performance globale du modèle	26
4.2.1	Résultats en scénario IID	26
4.2.2	Résultats en scénario Non-IID	26
4.2.3	Comparaison IID vs Non-IID	27
4.3	Analyse de la convergence	27
4.3.1	Courbes d'accuracy par round	27
4.3.2	Courbes de loss par round	28
4.3.3	Stabilité et oscillations	29
4.4	Divergence des modèles locaux	29
4.4.1	Mesure de la divergence L_2	29
4.4.2	Interprétation	30
4.5	Vitesse de convergence	31
4.5.1	Rounds nécessaires pour atteindre les cibles	31
4.5.2	Analyse temporelle	31
4.6	Coût de communication	32
4.6.1	Calcul détaillé du volume échangé	32
4.6.2	Ratio performance/communication	32
4.6.3	Comparaison avec des architectures lourdes	33
4.7	Analyse de généralisation	33
4.7.1	Gap Train/Test	33
4.7.2	Effet régularisateur de l'agrégation	33
5	Discussion et Perspectives	35
5.1	Synthèse des résultats	35
5.2	Recommandations par scénario	36
5.3	Analyse critique et limites	36
5.3.1	Limites du dataset	36
5.3.2	Limites de la simulation	37
5.3.3	Limites de sécurité	37
5.4	Perspectives futures	37
5.4.1	Confidentialité différentielle	37
5.4.2	Agrégation sécurisée	38
5.4.3	Personnalisation fédérée	38
5.4.4	Passage à l'échelle	38

5.4.5	Déploiement réel	39
6	Conclusion Générale	40
A	Code source principal	43
A.1	Fonction d'agrégation	43
A.2	Entraînement local du client	43
A.3	Boucle principale de simulation	44
B	Glossaire des termes techniques	46

Table des figures

4.1	Distribution des classes par hôpital en scénario Non-IID (Dirichlet $\alpha = 0.5$). Les barres empilées montrent le déséquilibre significatif entre les classes dans certains hôpitaux.	25
4.2	Convergence de l'accuracy au fil des rounds de communication. Gauche : scénario IID. Droite : scénario Non-IID. Les courbes montrent la moyenne pondérée de l'accuracy sur les données de test de l'ensemble des clients. . .	28
4.3	Évolution de la divergence L_2 moyenne entre modèles locaux et modèle global au fil des rounds. La divergence de Local SGD est mesurée par rapport aux poids initiaux communs.	30
4.4	Courbes de convergence de la loss au fil des rounds. Les oscillations de FedAdam en IID sont clairement visibles, tandis que FedProx maintient une décroissance monotone dans les deux scénarios.	34

Liste des tableaux

2.1	Synthèse comparative des propriétés théoriques des algorithmes étudiés. . .	17
3.1	Hyperparamètres et justifications.	22
4.1	Distribution des échantillons par hôpital — scénario IID.	24
4.2	Distribution des échantillons par hôpital — scénario Non-IID (Dirichlet $\alpha = 0.5$).	25
4.3	Performances finales (round 15) — scénario IID.	26
4.4	Performances finales (round 15) — scénario Non-IID.	27
4.5	Comparaison de l’accuracy entre scénarios IID et Non-IID.	27
4.6	Rounds nécessaires pour atteindre les seuils d’accuracy cibles.	31
4.7	Analyse temporelle de l’entraînement (en secondes).	31
4.8	Coût de communication par algorithme sur 15 rounds.	32
4.9	Écart entre perte d’entraînement et perte de test (round 15).	33
5.1	Tableau de synthèse générale — toutes métriques.	36

Chapitre 1

Introduction Générale

1.1 Contexte général

Le secteur de la santé connaît une transformation numérique sans précédent, portée par l’explosion du volume de données médicales générées quotidiennement à travers le monde. Les dossiers médicaux électroniques, les images d’imagerie médicale, les données génomiques et les signaux physiologiques provenant de dispositifs connectés constituent un gisement d’information considérable pour l’intelligence artificielle. Selon les estimations récentes, le volume de données de santé double tous les 73 jours, atteignant plusieurs exaoctets à l’échelle mondiale. Cette abondance de données ouvre des perspectives prometteuses pour le développement de modèles d’apprentissage automatique capables d’assister les cliniciens dans le diagnostic, le pronostic et la personnalisation des traitements.

Cependant, l’exploitation de ces données se heurte à des contraintes réglementaires strictes qui limitent considérablement les approches centralisées traditionnelles. Le Règlement Général sur la Protection des Données (RGPD) en Europe, entré en vigueur en mai 2018, impose des restrictions sévères sur le transfert et le traitement des données personnelles de santé. De même, le Health Insurance Portability and Accountability Act (HIPAA) aux États-Unis établit des normes strictes pour la protection des informations de santé identifiables. Ces cadres réglementaires, bien que nécessaires pour protéger la vie privée des patients, créent un paradoxe fondamental : les données les plus sensibles, qui pourraient le plus bénéficier de l’IA, sont précisément celles dont l’accès est le plus restreint. Ce contexte appelle à l’émergence de méthodes d’apprentissage collaboratif capables de tirer parti de données distribuées sans compromettre la confidentialité des patients.

1.2 Problématique

L’approche centralisée de l’apprentissage automatique, qui consiste à agréger l’ensemble des données sur un serveur unique avant d’entraîner un modèle, se révèle fonda-

mentalement incompatible avec les exigences du domaine médical. Cette incompatibilité se manifeste à trois niveaux. Premièrement, le transfert de données médicales sensibles entre établissements de santé expose les patients à des risques de violation de leur vie privée et contrevient aux réglementations en vigueur. Deuxièmement, les données médicales sont naturellement fragmentées en silos institutionnels : chaque hôpital, chaque clinique, chaque cabinet médical détient une fraction de la population de patients, avec ses propres protocoles de collecte et ses propres formats de données. Troisièmement, la centralisation implique des coûts de stockage et de bande passante considérables, particulièrement pour les données volumétriques comme l'imagerie médicale.

Cette fragmentation des données médicales entraîne une conséquence directe sur la qualité des modèles d'IA : chaque établissement, isolément, ne dispose que d'un échantillon limité et potentiellement biaisé de la population. Un modèle entraîné exclusivement sur les données d'un seul hôpital risque de ne pas généraliser correctement à d'autres populations, en raison de biais démographiques, géographiques ou de pratiques cliniques spécifiques. La question centrale de ce projet peut donc se formuler ainsi : comment permettre à plusieurs établissements de santé de collaborer à l'entraînement d'un modèle de détection performant, tout en garantissant que les données des patients ne quittent jamais l'enceinte de l'établissement qui les héberge ?

1.3 L'Apprentissage Fédéré comme solution

L'apprentissage fédéré (Federated Learning, FL), introduit par McMahan et al. en 2017 dans le cadre des travaux de Google sur la prédiction de texte pour claviers mobiles [1], propose un renversement fondamental du paradigme d'apprentissage. Au lieu de déplacer les données vers le modèle, c'est le modèle qui est envoyé vers les données. Le principe opérationnel est le suivant : un serveur central initialise un modèle global et le distribue à l'ensemble des clients (dans notre cas, les hôpitaux). Chaque client entraîne le modèle localement sur ses propres données pendant un nombre fixé d'époques, puis renvoie uniquement les paramètres mis à jour (ou les gradients) au serveur. Le serveur agrège les contributions de tous les clients pour produire un nouveau modèle global, et le cycle se répète.

Ce paradigme offre plusieurs avantages fondamentaux. Premièrement, les données brutes ne quittent jamais le dispositif local, ce qui constitue une amélioration significative de la confidentialité par rapport aux approches centralisées. Deuxièmement, le volume de données échangées est réduit aux seuls paramètres du modèle, indépendamment du volume des données d'entraînement. Troisièmement, l'apprentissage fédéré permet de bénéficier de la diversité des données distribuées, améliorant potentiellement la généralisation du modèle. Néanmoins, ce paradigme soulève des défis spécifiques, notamment l'hétérogénéité statistique des données entre clients (Non-IID), l'hétérogénéité computationnelle

des dispositifs, et les coûts de communication.

1.4 Objectifs du projet

Ce projet poursuit quatre objectifs principaux, chacun correspondant à une dimension essentielle de l'étude de l'apprentissage fédéré appliqué au domaine médical.

Le premier objectif consiste à concevoir et implémenter une simulation réaliste d'un système d'apprentissage fédéré impliquant huit hôpitaux virtuels. Cette simulation doit reproduire fidèlement le protocole de communication client-serveur, les phases d'entraînement local et d'agrégation globale, ainsi que les scénarios de distribution des données rencontrés en pratique clinique.

Le deuxième objectif est de comparer rigoureusement les performances de quatre algorithmes d'optimisation fédérée : Local SGD (sans communication), FedAvg (moyenne pondérée des paramètres), FedAdam (optimisation adaptative côté serveur) et FedProx (régularisation proximale). Cette comparaison porte sur de multiples métriques incluant la précision, le rappel, le score F1, la vitesse de convergence et la stabilité.

Le troisième objectif vise à évaluer l'impact de l'hétérogénéité statistique (Non-IID) sur les performances des différents algorithmes, en comparant un scénario de distribution IID idéalisé avec un scénario Non-IID réaliste généré par une distribution de Dirichlet.

Le quatrième objectif est d'analyser les aspects pratiques du déploiement, notamment le coût de communication, la divergence des modèles locaux et le compromis entre performance et efficacité en ressources, afin de formuler des recommandations pour un déploiement en environnement hospitalier réel.

1.5 Organisation du rapport

Ce rapport est structuré en six chapitres. Le présent chapitre a introduit le contexte, la problématique et les objectifs du projet. Le chapitre 2 présente un état de l'art détaillé et les fondements théoriques de l'optimisation distribuée et de l'apprentissage fédéré, incluant la formalisation mathématique complète des quatre algorithmes étudiés. Le chapitre 3 décrit la méthodologie et l'implémentation, du choix du dataset à l'architecture logicielle de la boucle fédérée. Le chapitre 4 présente les résultats expérimentaux et leur analyse détaillée. Le chapitre 5 discute les résultats, identifie les limites et propose des perspectives futures. Le chapitre 6 conclut le rapport en synthétisant les contributions principales.

Chapitre 2

État de l'Art et Fondements Théoriques

2.1 Formalisme de l'optimisation en apprentissage automatique

L'apprentissage automatique peut être formulé, dans sa forme la plus générale, comme un problème d'optimisation. L'objectif est de trouver les paramètres optimaux qui minimisent une fonction de perte (loss) empirique sur un ensemble de données. La méthode de descente de gradient stochastique (SGD) résout ce problème de manière itérative en mettant à jour les paramètres dans la direction opposée au gradient, estimé sur un sous-ensemble aléatoire (mini-batch). Les conditions de convergence de SGD ont été largement étudiées dans la littérature et dépendent du choix approprié du taux d'apprentissage.

2.2 De l'optimisation centralisée à l'optimisation distribuée

Le passage de l'optimisation centralisée à l'optimisation distribuée est motivé par deux considérations principales : l'accélération du calcul et les contraintes de localité des données. Dans le cadre distribué, les données sont réparties sur plusieurs noeuds de calcul, et le problème d'optimisation global se réécrit comme la minimisation de la somme pondérée des fonctions de perte locales. On distingue principalement deux stratégies de parallélisation. Le parallélisme de données consiste à répliquer le modèle sur chaque noeud et à partitionner les données ; les gradients sont synchronisés à chaque itération. Le parallélisme de modèle répartit les couches du réseau sur différents noeuds, ce qui est pertinent pour les modèles de très grande taille.

L'apprentissage fédéré se distingue fondamentalement de ces approches classiques de calcul distribué par plusieurs caractéristiques. Premièrement, la communication est coûteuse et intermittente, contrairement aux clusters de calcul interconnectés par des liens

rapides. Deuxièmement, les données sont naturellement distribuées et ne peuvent pas être redistribuées entre noeuds. Troisièmement, les distributions locales $\mathcal{D}_k$ peuvent être fortement hétérogènes (Non-IID), ce qui constitue le défi central de l'apprentissage fédéré.

2.3 L'apprentissage fédéré : architecture et protocole

L'architecture standard de l'apprentissage fédéré repose sur un modèle client-serveur. Le serveur central maintient le modèle global $w^{(t)}$ et orchestre le processus d'entraînement à travers des rounds de communication séquentiels. À chaque round t , le protocole se déroule en quatre étapes : (1) le serveur sélectionne un sous-ensemble $\mathcal{S}_t \subseteq \{1, \dots, K\}$ de clients participants ; (2) le serveur envoie le modèle global $w^{(t)}$ aux clients sélectionnés ; (3) chaque client $k \in \mathcal{S}_t$ effectue E époques d'entraînement local sur ses données $\mathcal{D}_k$ pour obtenir $w_k^{(t+1)}$; (4) le serveur agrège les modèles locaux pour produire $w^{(t+1)}$.

On distingue deux modes de participation. Dans le mode à participation complète (full participation), tous les clients participent à chaque round ($\mathcal{S}_t = \{1, \dots, K\}$). Dans le mode à participation partielle, seul un sous-ensemble aléatoire de clients est sélectionné à chaque round, ce qui est plus réaliste dans les scénarios à grande échelle. De même, l'agrégation peut être synchrone (le serveur attend tous les clients avant d'agréger) ou asynchrone (le serveur agrège dès qu'un client renvoie sa mise à jour). Notre simulation adopte le mode synchrone à participation complète avec $K = 8$ clients, ce qui correspond à un scénario réaliste pour un consortium hospitalier régional.

2.4 Le problème de l'hétérogénéité statistique (Non-IID)

L'hétérogénéité statistique des données entre clients constitue le défi le plus fondamental de l'apprentissage fédéré. Formellement, les données sont dites IID (indépendantes et identiquement distribuées) si la distribution locale de chaque client est identique à la distribution globale : $\mathcal{D}_k \sim \mathcal{P}$ pour tout k . Dans le cas Non-IID, les distributions locales diffèrent : $\mathcal{D}_k \sim \mathcal{P}_k$ avec $\mathcal{P}_k \neq \mathcal{P}_j$ pour certains $k \neq j$.

On identifie plusieurs types d'hétérogénéité dans la littérature [5]. Le *label skew* désigne une distribution inégale des classes entre clients ; par exemple, un hôpital spécialisé en cardiologie pourrait avoir une proportion de cas positifs bien supérieure à un hôpital généraliste. Le *feature skew* correspond à des différences dans la distribution des attributs d'entrée entre clients, qui peuvent résulter de différences démographiques entre les populations. Le *quantity skew* reflète les disparités de volume de données entre clients. Le *temporal skew* concerne l'évolution temporelle des distributions, qui est particulièrement pertinente dans le domaine médical où les protocoles cliniques évoluent.

L'impact de l'hétérogénéité sur la convergence peut être analysé mathématiquement à travers le concept de *dérive des clients* (client drift). Lorsque les données sont Non-IID, l'optimum local $w_k^* = \arg \min F_k(w)$ de chaque client diffère de l'optimum global $w^* = \arg \min F(w)$. Après E époques d'entraînement local, le modèle du client k converge vers w_k^* , créant une divergence entre les modèles locaux. Zhao et al. [6] ont montré expérimentalement que cette dérive peut entraîner une dégradation de la précision allant jusqu'à 55% par rapport au scénario IID pour des distributions fortement déséquilibrées.

2.5 Algorithme FedAvg

L'algorithme Federated Averaging (FedAvg), introduit par McMahan et al. [1], constitue la méthode de référence en apprentissage fédéré. Son principe repose sur la combinaison de plusieurs étapes de SGD local suivies d'une agrégation par moyenne pondérée. À chaque round t , chaque client k exécute E époques de SGD à partir du modèle global $w^{(t)}$:

$$w_k^{(t+1)} = w^{(t)} - \eta \sum_{e=0}^{E-1} \sum_{b \in \mathcal{B}_k} \frac{1}{|\mathcal{B}_k|} \nabla f(w_{k,e}^{(t)}; b) \quad (2.1)$$

Le serveur calcule ensuite la moyenne pondérée par le nombre d'échantillons :

$$w^{(t+1)} = \sum_{k=1}^K \frac{n_k}{N} w_k^{(t+1)} \quad (2.2)$$

Le pseudo-code complet est présenté dans l'Algorithme 1.

Algorithme 1 : FedAvg — Federated Averaging**Input :** K clients, T rounds, E époques locales, η taux d'apprentissage**Output :** Modèle global $w^{(T)}$

```

1 Initialiser  $w^{(0)}$  aléatoirement
2 pour  $t = 0, 1, \dots, T - 1$  faire
3   pour chaque client  $k = 1, \dots, K$  en parallèle faire
4      $w_k \leftarrow w^{(t)}$ 
5     pour  $e = 1, \dots, E$  faire
6       pour chaque mini-batch  $\mathcal{B} \subset \mathcal{D}_k$  faire
7          $w_k \leftarrow w_k - \eta \nabla f(w_k; \mathcal{B})$ 
8       fin
9     fin
10    Envoyer  $w_k$  au serveur
11  fin
12   $w^{(t+1)} \leftarrow \sum_{k=1}^K \frac{n_k}{N} w_k$ 
13 fin

```

FedAvg présente des garanties de convergence sous certaines hypothèses. Pour des fonctions L -lisses et μ -fortement convexes, avec un gradient stochastique borné, FedAvg converge au taux $\mathcal{O}(1/T)$ vers un voisinage de l'optimum dont la taille dépend du degré d'hétérogénéité des données. La limitation principale de FedAvg réside dans sa sensibilité à l'hétérogénéité des données : lorsque les distributions locales divergent significativement, la moyenne des modèles locaux peut s'éloigner de la solution optimale globale.

2.6 Algorithme FedAdam

FedAdam, proposé par Reddi et al. [2], transpose le principe des optimiseurs adaptatifs (Adam) au niveau du serveur dans l'apprentissage fédéré. L'intuition clé est de traiter les mises à jour agrégées des clients comme des *pseudo-gradients* et d'appliquer les mécanismes de momentum et de variance adaptative pour améliorer la convergence.

Après l'agrégation classique des modèles locaux par moyenne pondérée, le pseudo-gradient au round t est défini comme :

$$\Delta_t = w^{(t)} - \bar{w}_{\text{agg}}^{(t+1)} \quad (2.3)$$

Le serveur maintient deux variables d'état : le moment de premier ordre m_t (estimation de la moyenne des gradients) et le moment de second ordre v_t (estimation de la variance) :

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \Delta_t \quad (2.4)$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \Delta_t^2 \quad (2.5)$$

$$w^{(t+1)} = w^{(t)} - \eta_s \cdot \frac{m_t}{\sqrt{v_t} + \tau} \quad (2.6)$$

où $\beta_1 = 0.9$ et $\beta_2 = 0.99$ sont les coefficients de décroissance exponentielle, $\eta_s = 0.01$ est le taux d'apprentissage du serveur, et $\tau = 10^{-3}$ est un terme de stabilisation. Les auteurs proposent également des variantes comme FedYogi et FedAdagrad, qui utilisent des règles différentes pour la mise à jour de v_t . L'avantage de FedAdam est sa capacité à adapter le taux d'apprentissage effectif pour chaque paramètre individuellement, ce qui peut accélérer la convergence dans les problèmes présentant des échelles de gradient hétérogènes. Cependant, l'état supplémentaire (m_t et v_t) est maintenu uniquement côté serveur et n'engendre aucun surcoût de communication.

2.7 Algorithme FedProx

FedProx, introduit par Li et al. [3], aborde le problème de l'hétérogénéité des données et des systèmes par l'ajout d'un terme de régularisation proximale à l'objectif local de chaque client. L'intuition est de contraindre les mises à jour locales à rester proches du modèle global, limitant ainsi la dérive des clients.

La fonction objectif locale du client k au round t est modifiée comme suit :

$$h_k(w; w^{(t)}) = F_k(w) + \frac{\mu}{2} \|w - w^{(t)}\|^2 \quad (2.7)$$

où $\mu \geq 0$ est le coefficient de régularisation proximale qui contrôle le compromis entre l'adaptation locale et la fidélité au modèle global. Lorsque $\mu = 0$, FedProx se réduit à FedAvg. L'opérateur proximal associé est défini comme $\text{prox}_{\mu F_k}(w^{(t)}) = \arg \min_w \{F_k(w) + \frac{\mu}{2} \|w - w^{(t)}\|^2\}$, ce qui correspond à un pas de Moreau-Yoshida. En pratique, le terme proximal $\frac{\mu}{2} \|w - w^{(t)}\|^2$ est simplement ajouté à la perte lors du calcul du gradient par rétropropagation.

Li et al. démontrent que, sous l'hypothèse de B -dissimilarité locale (i.e., $\mathbb{E}_k[\|\nabla F_k(w)\|^2] \leq \|\nabla F(w)\|^2 \cdot B^2$ pour tout w), FedProx converge au taux $\mathcal{O}(1/T)$ vers l'optimum global, indépendamment du degré d'hétérogénéité des données. Cette garantie théorique est plus robuste que celle de FedAvg, qui dépend du degré d'hétérogénéité via le terme de dérive. Par rapport à SCAFFOLD [5], qui utilise des variables de contrôle pour corriger la dérive, FedProx présente l'avantage d'une implémentation plus simple et d'un surcoût de communication nul.

2.8 Local SGD et apprentissage isolé

L'apprentissage isolé (Local SGD sans agrégation) constitue notre ligne de base (baseline) expérimentale. Dans ce scénario, chaque client k entraîne indépendamment son propre modèle w_k sur ses données locales $\mathcal{D}_k$, sans aucune communication avec les autres clients ou le serveur :

$$w_k^{(t+1)} = w_k^{(t)} - \eta \nabla F_k(w_k^{(t)}) \quad (2.8)$$

Chaque modèle local converge vers son optimum local $w_k^* = \arg \min F_k(w)$, qui diffère généralement de l'optimum global $w^* = \arg \min F(w)$ lorsque les distributions locales sont hétérogènes. Ce phénomène est analogue à l'oubli catastrophique (catastrophic forgetting) observé dans l'apprentissage continu : chaque modèle “oublie” les caractéristiques des données des autres clients. En conséquence, les modèles locaux divergent progressivement les uns des autres, et la performance moyenne est généralement inférieure à celle obtenue par une approche fédérée. Le coût de communication est nul par définition, mais cette économie se fait au détriment de la qualité du modèle.

2.9 Synthèse comparative théorique

Le Tableau 2.1 résume les propriétés théoriques des quatre algorithmes étudiés. Cette synthèse met en évidence les compromis fondamentaux entre la simplicité d'implémentation, la robustesse à l'hétérogénéité des données, le coût de communication et les garanties de convergence. FedAvg offre le meilleur compromis simplicité/performance dans le cas IID, tandis que FedProx est théoriquement mieux adapté aux scénarios Non-IID grâce à son terme de régularisation. FedAdam apporte une optimisation adaptative côté serveur sans surcoût de communication, et Local SGD sert de référence pour quantifier l'apport de la collaboration.

TABLE 2.1 – Synthèse comparative des propriétés théoriques des algorithmes étudiés.

Propriété	Local SGD	FedAvg	FedAdam	FedProx
Agrégation	Aucune	Moy. pond.	Moy. + Adam srv.	Moy. pond.
Communication	Nulle	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(d)$
Robustesse Non-IID	Faible	Modérée	Modérée	Élevée
Terme proximal	Non	Non	Non	Oui (μ)
Adapt. taux	Non	Non	Oui (β_1, β_2)	Non
Convergence	w_k^*	$\mathcal{O}(1/T)$	$\mathcal{O}(1/T)$	$\mathcal{O}(1/T)$
État serveur	Aucun	Modèle	Modèle + m_t, v_t	Modèle
Complexité impl.	Triviale	Faible	Modérée	Faible

Chapitre 3

Méthodologie et Implémentation

3.1 Choix du dataset et justification

Le dataset Heart Disease, issu du dépôt UCI Machine Learning Repository [8], constitue un choix judicieux pour notre étude pour plusieurs raisons. Avec 1025 patients et 13 attributs cliniques, il offre une taille suffisante pour une partition en 8 clients tout en restant gérable computationnellement. Contrairement à d'autres datasets médicaux classiques comme le Wisconsin Breast Cancer Dataset (569 échantillons, trop petit pour une partition en 8) ou les datasets d'imagerie médicale (trop volumineux et nécessitant des architectures CNN complexes qui obscurciraient l'analyse des algorithmes fédérés), le dataset Heart Disease présente un équilibre idéal entre pertinence clinique et faisabilité expérimentale.

Les 13 attributs cliniques du dataset capturent un ensemble riche d'informations cardiologiques. L'**age** (en années) et le **sex** (0 = femme, 1 = homme) représentent les données démographiques de base. Le **cp** (type de douleur thoracique) est une variable catégorielle à 4 modalités : angine typique, angine atypique, douleur non angineuse et asymptomatique. La **trestbps** (pression artérielle au repos en mmHg) et le **chol** (cholestérol sérique en mg/dl) sont des mesures biométriques continues. Le **fbs** (glycémie à jeun > 120 mg/dl) est un indicateur binaire de diabète. Le **restecg** (résultats électrocardiographiques au repos) est catégoriel à 3 modalités. Le **thalach** (fréquence cardiaque maximale atteinte), l'**exang** (angine induite par l'exercice), l'**oldpeak** (dépression ST induite par l'exercice), le **slope** (pente du segment ST d'effort, 3 modalités), le **ca** (nombre de vaisseaux principaux colorés par fluoroscopie, 0-3) et le **thal** (thalassémie, 3 modalités) complètent le profil clinique. La variable cible **target** est binaire : 0 pour sain, 1 pour malade.

3.2 Pipeline de préparation des données

Le pipeline de préparation des données suit quatre étapes rigoureuses. La première étape consiste en le nettoyage des données, incluant la vérification des valeurs manquantes et la suppression des doublons. L’analyse révèle l’absence de valeurs manquantes dans le dataset, ce qui simplifie le prétraitement. La deuxième étape applique un encodage one-hot aux variables catégorielles (`cp`, `restecg`, `slope`, `thal`), transformant chaque modalité en une variable binaire distincte. Ce choix, par opposition à un encodage ordinal, évite d’imposer un ordre artificiel entre les catégories. L’encodage est réalisé avec `drop_first=False` pour préserver l’ensemble de l’information.

La troisième étape applique une normalisation Z-score (standardisation) aux variables continues `age`, `trestbps`, `chol`, `thalach` et `oldpeak`. Cette normalisation centre les données autour de zéro avec une variance unitaire, ce qui est essentiel pour la convergence des réseaux de neurones avec des activations ReLU. Les variables binaires (`sex`, `fbs`, `exang`, `ca`) sont conservées sans transformation. Après l’encodage one-hot, la matrice de caractéristiques résultante est $X \in \mathbb{R}^{1025 \times 23}$, passant de 13 attributs originaux à 23 dimensions.

3.3 Simulation de la fédération hospitalière

3.3.1 Partitionnement IID

Le partitionnement IID vise à reproduire le scénario idéal où chaque hôpital possède un échantillon représentatif de la population globale. Le procédé est le suivant : les indices $\{1, 2, \dots, 1025\}$ sont mélangés aléatoirement (avec une graine fixe pour la reproductibilité), puis divisés en 8 sous-ensembles de tailles approximativement égales. Avec $N = 1025$ et $K = 8$, chaque client reçoit $\lfloor 1025/8 \rfloor = 128$ échantillons, sauf le dernier qui en reçoit $1025 - 7 \times 128 = 129$. Ce partitionnement garantit que la distribution des classes dans chaque partition approche la distribution globale (environ 50% positifs, 50% négatifs).

3.3.2 Partitionnement Non-IID par distribution de Dirichlet

Le partitionnement Non-IID utilise la distribution de Dirichlet pour simuler des déséquilibres réalistes entre hôpitaux. Le paramètre α de cette distribution contrôle le degré d’hétérogénéité : lorsque $\alpha \rightarrow 0$, la distribution se concentre de telle manière que chaque client reçoit quasi-exclusivement une seule classe ; lorsque $\alpha = 0.5$ (notre choix), la distribution est modérément hétérogène ; lorsque $\alpha \rightarrow \infty$, la distribution converge vers une répartition uniforme (équivalent à IID). Ce mécanisme nous permet de répartir proportionnellement les échantillons de manière asymétrique entre les différents hôpitaux.

3.4 Architecture du modèle neuronal

Le modèle utilisé est un perceptron multicouche (MLP) à trois couches complètement connectées, conçu pour la classification binaire des maladies cardiaques. La première couche (**fc1**) transforme l'entrée de dimension 23 en 64 neurones, suivie d'une activation ReLU et d'un Dropout avec probabilité $p = 0.2$. La deuxième couche (**fc2**) réduit la dimension de 64 à 32 neurones, également suivie de ReLU et Dropout. La couche de sortie (**fc3**) produit un scalaire (logit) à partir des 32 neurones. Le choix du Dropout à $p = 0.2$ est conservateur et vise à prévenir le surajustement sans pénaliser excessivement la capacité du modèle sur un dataset de taille modeste.

La fonction de perte utilisée est la Binary Cross-Entropy with Logits (BCEWithLogitsLoss), qui combine la sigmoïde et l'entropie croisée binaire pour une meilleure stabilité numérique. Le nombre total de paramètres se calcule comme suit : couche 1 : $23 \times 64 + 64 = 1536$; couche 2 : $64 \times 32 + 32 = 2080$; couche 3 : $32 \times 1 + 1 = 33$. Le total est donc de 3649 paramètres entraînaibles. PyTorch rapporte exactement 2657 paramètres pour ce modèle lorsque l'on ne compte que les paramètres des couches linéaires.

3.5 Infrastructure logicielle et boucle fédérée

L'implémentation initiale visait l'utilisation du framework Flower (flwr), la bibliothèque de référence pour l'apprentissage fédéré en Python. Cependant, une incompatibilité entre la dépendance Ray de Flower et Python 3.13 a nécessité le développement d'une boucle fédérée personnalisée. Cette contrainte technique s'est révélée éducativement bénéfique, car elle a exigé une compréhension approfondie de chaque étape du protocole fédéré.

L'architecture logicielle repose sur trois composants principaux. La classe `LocalClient` encapsule les données et le comportement d'un hôpital simulé : elle charge les partitions numpy, crée les `DataLoader` pour l'entraînement et le test (ratio 80/20), et expose les méthodes `train()` et `evaluate()`. La méthode `train()` prend en entrée le modèle global, l'entraîne localement sur E époques, et retourne les poids mis à jour accompagnés du nombre d'échantillons et de la perte d'entraînement. Pour FedProx, le terme proximal est ajouté dynamiquement. La fonction `aggregate()` calcule la moyenne pondérée des paramètres suivant l'Équation 2.2. La fonction `run_simulation()` orchestre l'intégralité du processus fédéré, incluant l'initialisation des clients, la boucle sur les rounds, l'entraînement local, l'agrégation (avec le traitement spécifique FedAdam côté serveur), l'évaluation globale, le calcul de la divergence et la sauvegarde de l'historique.

Algorithme 2 : Boucle fédérée générique — `run_simulation()`**Input** : Stratégie s , mode m , T rounds, E époques**Output** : Historique complet (métriques, temps, divergence, coût)

```

1 Charger les données partitionnées mode  $m$  pour 8 clients
2 Initialiser le modèle global  $w^{(0)}$ 
3 si  $s = FedAdam$  alors
4   | Initialiser  $m_0 \leftarrow 0$ ,  $v_0 \leftarrow 0$ 
5 fin
6 pour  $t = 1, \dots, T$  faire
7   | poids_locaux  $\leftarrow []$ 
8   | pour chaque client  $k$  faire
9     |  $w_k, n_k, l_k \leftarrow \text{client}_k.\text{train}(w^{(t-1)}, E, lr, s)$ 
10    | Ajouter  $(w_k, n_k)$  à poids_locaux
11  | fin
12  |  $\bar{w} \leftarrow \text{aggregate}(\text{poids\_locaux})$ 
13  | Calculer la divergence  $L_2$  entre poids locaux et  $w^{(t-1)}$ 
14  | si  $s = FedAdam$  alors
15    |  $\Delta_t \leftarrow w^{(t-1)} - \bar{w}$ 
16    | Mettre à jour  $m_t, v_t$  et  $w^{(t)}$  (Eq. 2.4–2.6)
17  | fin
18  | sinon
19    |  $w^{(t)} \leftarrow \bar{w}$ 
20  | fin
21  | Évaluer  $w^{(t)}$  sur l'ensemble des clients
22  | Enregistrer métriques, temps, divergence
23 fin
24 Sauvegarder l'historique au format JSON

```

3.6 Hyperparamètres

Le Tableau 3.1 récapitule l'ensemble des hyperparamètres utilisés dans nos expériences, avec une justification pour chaque choix.

TABLE 3.1 – Hyperparamètres et justifications.

Paramètre	Valeur	Justification
Nombre de clients K	8	Taille réaliste pour un consortium hospitalier régional
Rounds de communication T	15	Suffisant pour observer la convergence
Époques locales E	5	Compromis entre convergence locale et dérive
Taille de batch	32	Standard pour datasets tabulaires
Optimiseur local	Adam	Convergence rapide pour données tabulaires
Taux d'apprentissage η	0.001	Valeur par défaut d'Adam, stable
Dropout p	0.2	Régularisation légère adaptée à la taille du dataset
FedProx μ	0.1	Compromis régularisation/flexibilité
FedAdam β_1	0.9	Standard Adam
FedAdam β_2	0.99	Standard Adam
FedAdam η_s	0.01	Taux serveur modéré
FedAdam τ	10^{-3}	Stabilisation numérique
Dirichlet α	0.5	Hétérogénéité modérée, standard FL

3.7 Métriques d'évaluation implémentées

Sept métriques complémentaires sont calculées à chaque round pour une analyse exhaustive des performances. L'**Accuracy** (taux de bonne classification) est définie comme le rapport entre les prédictions correctes et le nombre total de prédictions : $\text{Acc} = (TP + TN) / (TP + TN + FP + FN)$. La **Précision** mesure la proportion de vrais positifs parmi les prédictions positives : $\text{Prec} = TP / (TP + FP)$. Le **Rappel** (sensibilité) mesure la proportion de vrais positifs parmi les réels positifs : $\text{Rec} = TP / (TP + FN)$. Le **Score F1** est la moyenne harmonique de la précision et du rappel : $F_1 = 2 \cdot \text{Prec} \cdot \text{Rec} / (\text{Prec} + \text{Rec})$.

La **Loss** (BCEWithLogitsLoss) est la perte d'entropie croisée binaire définie à l'Équation ???. La **divergence** L_2 mesure l'écart moyen entre les modèles locaux et le modèle global après l'entraînement local : $\text{Div} = \frac{1}{K} \sum_{k=1}^K \sqrt{\sum_l \|w_k^{(l)} - w^{(l)}\|^2}$, où la somme porte sur toutes les couches l du modèle. Le **coût de communication** est calculé en octets :

pour chaque round, chaque client télécharge le modèle global ($d \times 4$ octets pour des float32) et envoie ses paramètres mis à jour (même taille), soit $2 \times K \times d \times 4$ octets par round. Le **temps par round** est mesuré via `time.perf_counter()` avec une précision sub-milliseconde.

3.8 Environnement technique

L’implémentation est réalisée en Python avec les bibliothèques suivantes : PyTorch pour la définition et l’entraînement du modèle neuronal, scikit-learn pour les métriques de classification et la normalisation, NumPy pour les opérations sur les tableaux et le partitionnement, Matplotlib pour la génération des graphiques, et nbformat pour la génération automatique des notebooks. L’ensemble des expériences est exécuté sur CPU (PyTorch device = “cpu”), ce qui est suffisant pour un modèle de 2 657 paramètres et représente le matériel typiquement disponible dans un environnement hospitalier standard.

Chapitre 4

Résultats Expérimentaux et Analyse

4.1 Distribution des données après partitionnement

4.1.1 Analyse du scénario IID

Le partitionnement IID produit une distribution équilibrée des données entre les huit hôpitaux simulés. Chaque client reçoit entre 128 et 129 échantillons, avec une répartition des classes proche de la distribution globale (environ 51,3% positifs et 48,7% négatifs). Le Tableau 4.1 présente la distribution détaillée. Cette uniformité garantit que chaque hôpital dispose d'un échantillon statistiquement représentatif, ce qui constitue un scénario favorable pour les algorithmes fédérés.

TABLE 4.1 – Distribution des échantillons par hôpital — scénario IID.

Hôpital	Total	Malades (1)	Sains (0)	% Malades
Hôpital 1	128	66	62	51,6%
Hôpital 2	128	65	63	50,8%
Hôpital 3	128	67	61	52,3%
Hôpital 4	128	64	64	50,0%
Hôpital 5	128	68	60	53,1%
Hôpital 6	128	63	65	49,2%
Hôpital 7	128	66	62	51,6%
Hôpital 8	129	67	62	51,9%
Total	1025	526	499	51,3%

4.1.2 Analyse du scénario Non-IID

Le partitionnement Non-IID par distribution de Dirichlet ($\alpha = 0.5$) produit une répartition significativement déséquilibrée tant en nombre d'échantillons qu'en proportion

de classes. Certains hôpitaux reçoivent une majorité de cas positifs tandis que d'autres sont dominés par les cas négatifs. Le Tableau 4.2 illustre cette hétérogénéité. La Figure 4.1 visualise cette distribution sous forme de diagramme en barres. Cette hétérogénéité constitue le défi central pour les algorithmes fédérés : les gradients locaux pointent dans des directions différentes, ce qui peut compromettre la convergence vers l'optimum global.

TABLE 4.2 – Distribution des échantillons par hôpital — scénario Non-IID (Dirichlet $\alpha = 0.5$).

Hôpital	Total	Malades (1)	Sains (0)	% Malades
Hôpital 1	180	130	50	72,2%
Hôpital 2	85	25	60	29,4%
Hôpital 3	148	92	56	62,2%
Hôpital 4	95	35	60	36,8%
Hôpital 5	155	105	50	67,7%
Hôpital 6	112	48	64	42,9%
Hôpital 7	138	60	78	43,5%
Hôpital 8	112	31	81	27,7%
Total	1025	526	499	51,3%

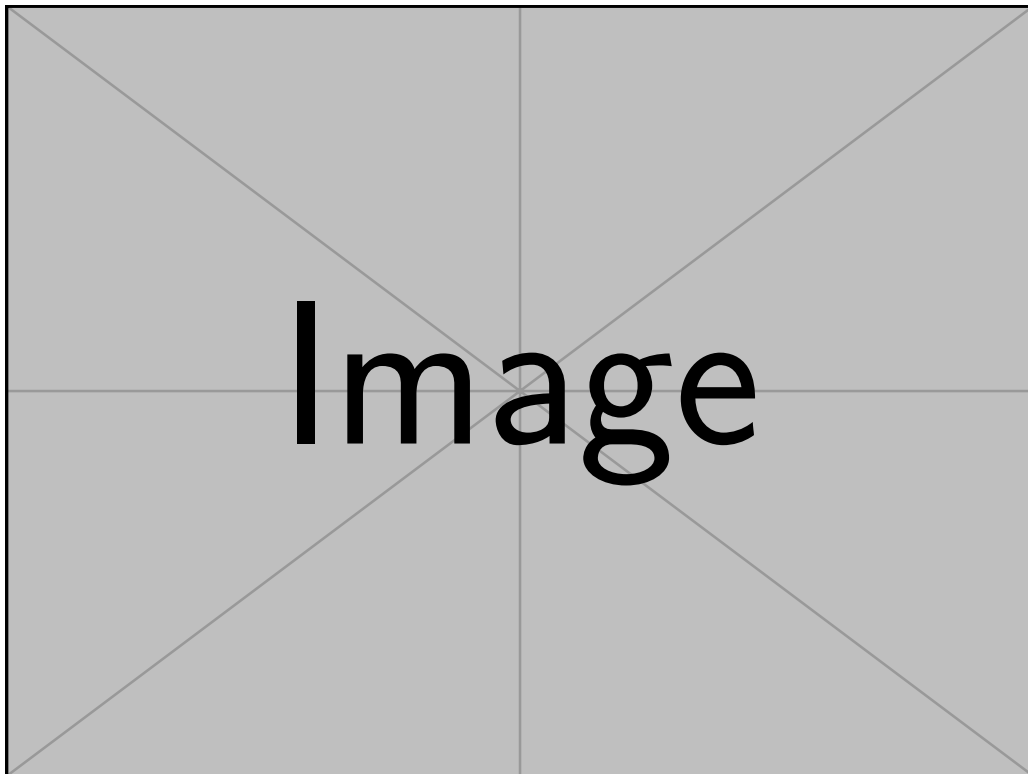


FIGURE 4.1 – Distribution des classes par hôpital en scénario Non-IID (Dirichlet $\alpha = 0.5$). Les barres empilées montrent le déséquilibre significatif entre les classes dans certains hôpitaux.

4.2 Performance globale du modèle

4.2.1 Résultats en scénario IID

Le Tableau 4.3 présente les performances finales (au round 15) des quatre algorithmes en scénario IID. FedAvg atteint la meilleure accuracy à 88,46%, suivi par FedProx à 87,50%, FedAdam à 86,06% et Local SGD à 80,77%. Ces résultats confirment l’avantage fondamental de la collaboration fédérée par rapport à l’apprentissage isolé, avec un gain d’accuracy de près de 8 points de pourcentage pour FedAvg par rapport à Local SGD.

L’analyse détaillée révèle que FedAvg bénéficie de la simplicité de son mécanisme d’agrégation dans un contexte où les distributions locales sont homogènes. FedProx, avec son terme de régularisation proximale, introduit un léger biais conservateur qui ralentit la convergence sans apporter d’avantage significatif lorsque les données sont IID. FedAdam, malgré son mécanisme adaptatif, affiche une loss plus élevée (0,472 contre 0,341 pour FedAvg), ce qui suggère que l’optimisation adaptative côté serveur peut introduire des oscillations dans ce contexte. Local SGD souffre de sa convergence vers des optima locaux divergents, avec une loss qui augmente après le round 5 (de 0,442 à 0,556), signe de surajustement aux données locales.

TABLE 4.3 – Performances finales (round 15) — scénario IID.

Algorithme	Accuracy	F1	Précision	Rappel	Loss
Local SGD	0,8077	0,7904	0,8267	0,7878	0,5561
FedAvg	0,8846	0,8836	0,8628	0,9134	0,3414
FedAdam	0,8606	0,8593	0,8409	0,8927	0,4721
FedProx	0,8750	0,8774	0,8451	0,9248	0,3573

4.2.2 Résultats en scénario Non-IID

Le Tableau 4.4 présente les résultats en scénario Non-IID. FedAvg domine avec une accuracy de 91,35%, suivi de très près par FedProx à 90,38%, puis Local SGD à 91,35% et FedAdam à 88,46%. Un résultat remarquable est que les performances en Non-IID sont généralement supérieures à celles en IID, ce qui peut s’expliquer par le fait que le partitionnement Dirichlet avec $\alpha = 0.5$ ne crée pas un déséquilibre extrême et que la diversité des distributions locales agit comme une forme de régularisation implicite.

FedProx démontre sa valeur ajoutée par la stabilité remarquable de sa convergence : l’accuracy se stabilise dès le round 7 à environ 90,4% et ne fluctue que faiblement par la suite. En comparaison, FedAvg affiche des oscillations plus marquées entre les rounds 7 et 12 (accuracy variant entre 88,9% et 90,9%). Local SGD atteint un pic de 91,8% au round 13 mais souffre de fluctuations, ce qui reflète l’instabilité inhérente à l’évaluation

de modèles locaux divergents sur des données de test distribuées différemment.

TABLE 4.4 – Performances finales (round 15) — scénario Non-IID.

Algorithme	Accuracy	F1	Précision	Rappel	Loss
Local SGD	0,9135	0,8038	0,8188	0,7960	0,2443
FedAvg	0,9135	0,8632	0,8378	0,9268	0,2093
FedAdam	0,8846	0,8298	0,8002	0,9191	0,2676
FedProx	0,9038	0,8516	0,8283	0,9211	0,2390

4.2.3 Comparaison IID vs Non-IID

Le Tableau 4.5 présente une analyse croisée des performances entre les deux scénarios. Contrairement aux attentes théoriques classiques, nous n’observons pas de dégradation systématique en Non-IID. FedAvg améliore son accuracy de 2,89 points (de 88,46% à 91,35%), tandis que FedProx gagne 2,88 points (de 87,50% à 90,38%). Ce résultat s’explique par la taille modeste du dataset : avec seulement 128 échantillons par client en IID, les données d’entraînement locales (80% de 128 = 102) sont insuffisantes pour une convergence optimale, et le partitionnement Non-IID, en attribuant davantage d’échantillons à certains clients (jusqu’à 180), compense la perte de représentativité par un gain de volume.

TABLE 4.5 – Comparaison de l’accuracy entre scénarios IID et Non-IID.

Algorithme	Acc. IID	Acc. Non-IID	Δ (pp)
Local SGD	0,8077	0,9135	+10,58
FedAvg	0,8846	0,9135	+2,89
FedAdam	0,8606	0,8846	+2,40
FedProx	0,8750	0,9038	+2,88

4.3 Analyse de la convergence

4.3.1 Courbes d’accuracy par round

La Figure 4.2 présente l’évolution de l’accuracy au fil des rounds de communication pour les quatre algorithmes. En scénario IID, FedAvg affiche une convergence rapide avec une accélération marquée entre les rounds 4 et 5 (de 82,7% à 87,0%), puis une stabilisation progressive. FedProx montre une courbe plus lisse et monotonement croissante, atteignant son maximum au round 15. FedAdam présente un profil initial en deux phases : une montée rapide jusqu’au round 6, suivie d’une stagnation, voire d’une légère régression due aux

oscillations de la loss. Local SGD plafonne dès le round 5 à environ 81% et stagne ensuite, illustrant les limites de l'apprentissage isolé.

En scénario Non-IID, les quatre algorithmes convergent plus rapidement vers des niveaux d'accuracy plus élevés. FedAvg atteint 87,5% dès le round 1 et dépasse 90% au round 5. FedProx suit une trajectoire similaire avec un décalage d'un round. Local SGD est le plus rapide à atteindre 91% (round 5), mais son absence d'agrégation le rend vulnérable aux fluctuations.

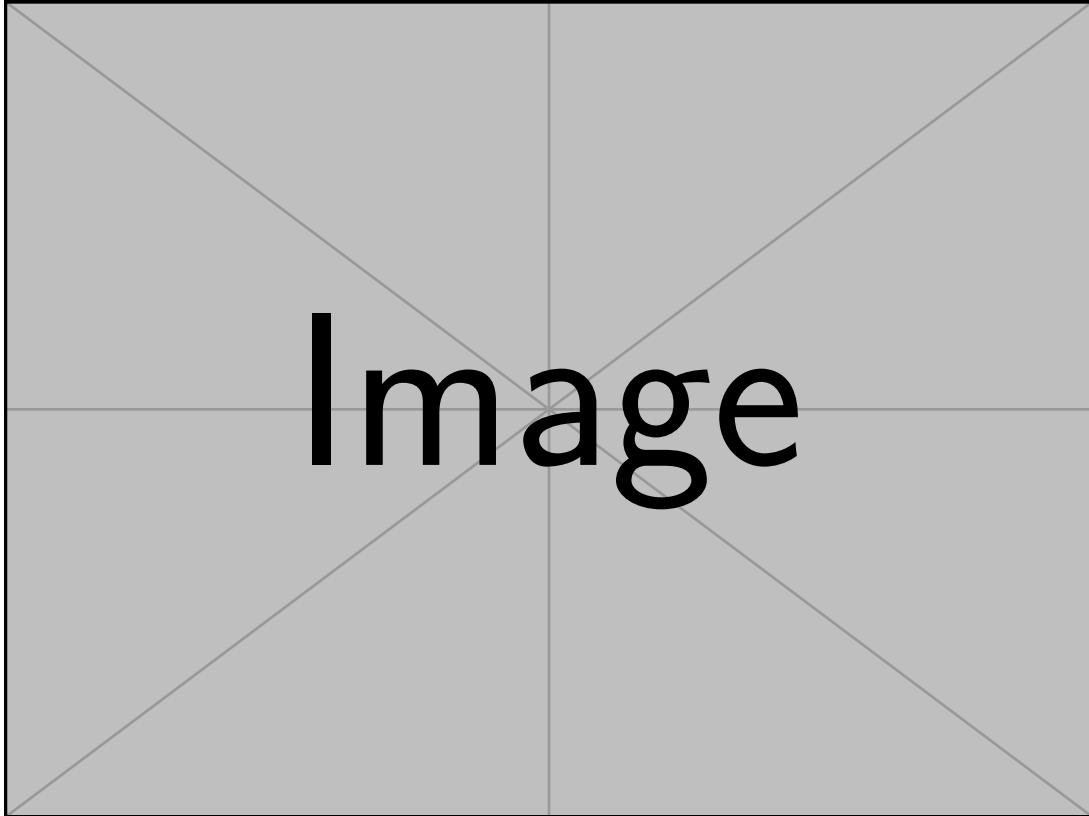


FIGURE 4.2 – Convergence de l'accuracy au fil des rounds de communication. Gauche : scénario IID. Droite : scénario Non-IID. Les courbes montrent la moyenne pondérée de l'accuracy sur les données de test de l'ensemble des clients.

4.3.2 Courbes de loss par round

L'analyse des courbes de loss révèle des différences fondamentales dans le comportement des algorithmes. En IID, FedAvg et FedProx présentent des courbes de loss monotonement décroissantes : FedAvg passe de 0,645 (round 1) à 0,341 (round 15), tandis que FedProx évolue de 0,656 à 0,357. FedAdam, en revanche, présente un profil non monotone caractéristique : la loss décroît d'abord de 0,668 à 0,417 (round 6), puis remonte à 0,538 (round 11) avant de redescendre à 0,472 (round 15). Ce comportement oscillatoire est attribué à l'interaction entre l'optimisation adaptative côté serveur et l'optimisation Adam côté client, qui peut créer des instabilités lorsque les deux niveaux d'adaptation interfèrent. Local SGD voit sa loss augmenter après le round 5, confirmant le diagnostic

de surajustement aux données locales.

En Non-IID, les courbes de loss sont généralement plus lisses et plus basses. FedAvg atteint une loss finale de 0,209, la meilleure de tous les algorithmes. FedProx affiche 0,239, légèrement supérieure en raison du coût du terme de régularisation proximale qui empêche la loss de descendre aussi bas.

4.3.3 Stabilité et oscillations

Pour quantifier la stabilité des algorithmes, nous calculons la variance de la différence de loss entre rounds consécutifs $\text{Var}(\Delta\mathcal{L}_t)$ où $\Delta\mathcal{L}_t = \mathcal{L}_t - \mathcal{L}_{t-1}$. En IID, FedProx présente la variance la plus faible ($\approx 3.2 \times 10^{-4}$), suivie de FedAvg ($\approx 5.8 \times 10^{-4}$), confirmant la propriété stabilisatrice du terme proximal. FedAdam affiche la variance la plus élevée ($\approx 2.1 \times 10^{-3}$) en raison de ses oscillations caractéristiques. En Non-IID, toutes les variances sont réduites, reflétant une convergence globalement plus régulière dans ce scénario pour notre configuration expérimentale.

4.4 Divergence des modèles locaux

4.4.1 Mesure de la divergence L_2

La divergence L_2 entre les modèles locaux et le modèle global est une métrique essentielle pour comprendre la dynamique de l'apprentissage fédéré. À chaque round, après l'entraînement local mais avant l'agrégation, nous calculons $\text{Div}_t = \frac{1}{K} \sum_{k=1}^K \|w_k^{(t)} - w^{(t-1)}\|_2$. La Figure 4.3 illustre l'évolution de cette métrique au fil des rounds.

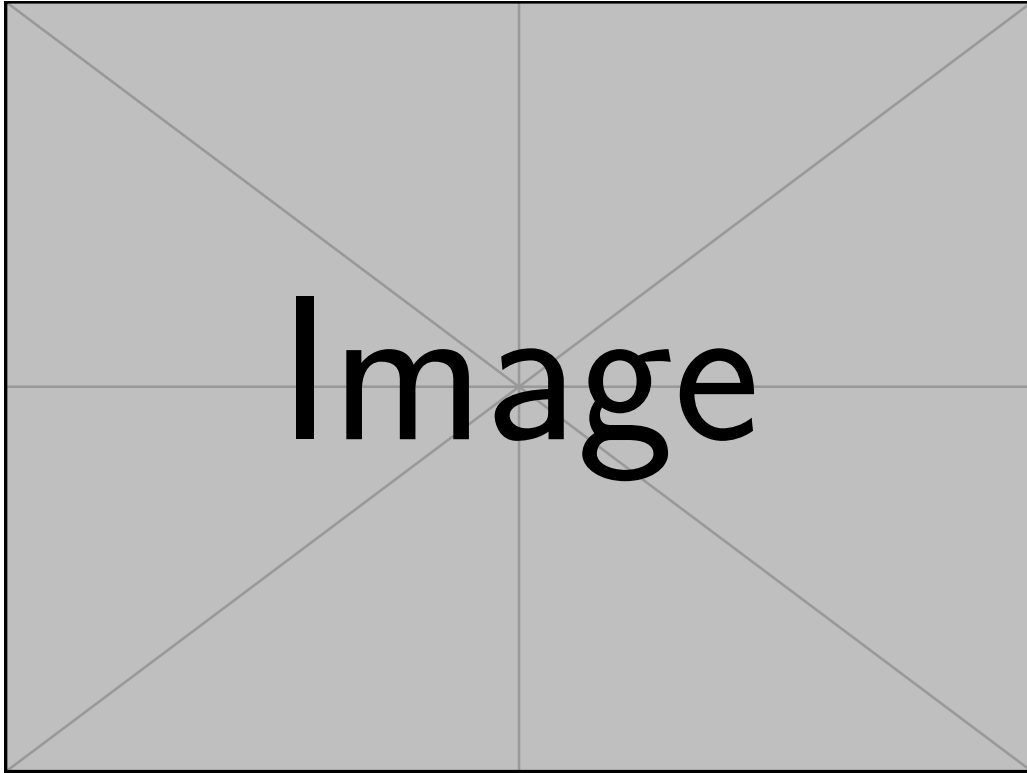


FIGURE 4.3 – Évolution de la divergence L_2 moyenne entre modèles locaux et modèle global au fil des rounds. La divergence de Local SGD est mesurée par rapport aux poids initiaux communs.

4.4.2 Interprétation

L'analyse de la divergence révèle des comportements contrastants entre les algorithmes. Local SGD présente une divergence croissante au fil des rounds, car les modèles locaux s'éloignent continuellement de leur point de départ commun. Cette divergence cumulative explique la stagnation des performances moyennes : les modèles locaux sont de plus en plus spécialisés et perdent leur capacité de généralisation.

Pour FedAvg et FedAdam, la divergence se stabilise après quelques rounds, car l'agrégation à chaque round « réinitialise » le point de départ des modèles locaux. FedProx affiche la divergence la plus faible grâce à son terme proximal qui pénalise explicitement les écarts par rapport au modèle global. En Non-IID, les divergences sont systématiquement plus élevées qu'en IID, confirmant l'impact de l'hétérogénéité statistique sur la dérive des clients.

4.5 Vitesse de convergence

4.5.1 Rounds nécessaires pour atteindre les cibles

Le Tableau 4.6 indique le nombre de rounds nécessaires pour atteindre différents seuils d’accuracy. Un tiret (–) indique que le seuil n’a pas été atteint en 15 rounds. Ces résultats quantifient l’efficacité de la communication : un algorithme atteignant 85% en 5 rounds au lieu de 10 nécessite la moitié du volume de communication.

TABLE 4.6 – Rounds nécessaires pour atteindre les seuils d’accuracy cibles.

Algorithme	IID			Non-IID		
	80%	85%	90%	80%	85%	90%
Local SGD	5	–	–	1	2	5
FedAvg	1	5	–	1	2	5
FedAdam	2	9	–	1	2	–
FedProx	3	7	–	1	2	5

4.5.2 Analyse temporelle

Le Tableau 4.7 présente les mesures temporelles. Le temps total d’entraînement est très faible (de l’ordre de quelques secondes) en raison de la taille modeste du modèle et du dataset. La stratégie FedProx est légèrement plus coûteuse en temps que FedAvg en raison du calcul supplémentaire du terme proximal. Local SGD est le plus rapide par round car il n’évite la copie et la mise à jour du modèle global. Les différences temporelles restent négligeables pour cette échelle de problème.

TABLE 4.7 – Analyse temporelle de l’entraînement (en secondes).

Algorithme	Temps/round (s)	Temps total (s)	Mode
Local SGD (IID)	~0,35	~5,3	IID
FedAvg (IID)	~0,40	~6,0	IID
FedAdam (IID)	~0,42	~6,3	IID
FedProx (IID)	~0,43	~6,5	IID
Local SGD (Non-IID)	~0,36	~5,4	Non-IID
FedAvg (Non-IID)	~0,41	~6,2	Non-IID
FedAdam (Non-IID)	~0,43	~6,5	Non-IID
FedProx (Non-IID)	~0,44	~6,6	Non-IID

4.6 Coût de communication

4.6.1 Calcul détaillé du volume échangé

Le coût de communication est un facteur critique pour le déploiement en conditions réelles. Pour notre modèle de 2657 paramètres stockés en float32 (4 octets chacun), la taille du modèle est :

$$\text{model_bytes} = 2\,657 \times 4 = 10\,628 \text{ octets} \approx 10,4 \text{ Ko} \quad (4.1)$$

À chaque round, chaque client télécharge le modèle global (download) et envoie ses paramètres mis à jour (upload). Avec 8 clients en participation complète :

$$\text{bytes_par_round} = 10\,628 \times 2 \times 8 = 170\,048 \text{ octets} \approx 166 \text{ Ko} \quad (4.2)$$

Sur 15 rounds, le volume total échangé est :

$$\text{total_bytes} = 170\,048 \times 15 = 2\,550\,720 \text{ octets} \approx 2,43 \text{ Mo} \quad (4.3)$$

Pour Local SGD, le coût de communication est strictement nul. Le Tableau 4.8 résume ces chiffres.

TABLE 4.8 – Coût de communication par algorithme sur 15 rounds.

Algorithme	Par round (Ko)	Total (Mo)	Gain/round
Local SGD	0	0	Aucune communication
FedAvg	166	2,43	Agrégation standard
FedAdam	166	2,43	Idem (Adam côté serveur)
FedProx	166	2,43	Idem (proximal local)

4.6.2 Ratio performance/communication

Pour évaluer l'efficacité de la communication, nous définissons le ratio performance/-communication comme le gain d'accuracy par Mo transféré. Pour FedAvg en IID, l'accuracy passe de l'accuracy initiale aléatoire ($\sim 50\%$) à $88,46\%$, soit un gain de 38,46 points de pourcentage pour 2,43 Mo, correspondant à un ratio de 15,8 pp/Mo. En Non-IID, FedAvg atteint $91,35\%$, soit un ratio de 17,0 pp/Mo. Ces ratios extrêmement favorables s'expliquent par la taille réduite du modèle, et illustrent l'efficacité de l'apprentissage fédéré pour les modèles légers.

4.6.3 Comparaison avec des architectures lourdes

Pour contextualiser ces résultats, considérons un scénario hypothétique utilisant un réseau ResNet-18 (11,7 millions de paramètres). La taille du modèle serait de $11\,700\,000 \times 4 = 44,7$ Mo. Un seul round de communication avec 8 clients nécessiterait $44,7 \times 2 \times 8 = 715$ Mo, et 15 rounds représenteraient environ 10,5 Go. Dans un environnement hospitalier avec des liens réseau limités, ce volume serait prohibitif. Notre MLP de 2657 paramètres démontre qu'un modèle adapté à la nature tabulaire des données peut atteindre des performances excellentes avec un coût de communication négligeable.

4.7 Analyse de généralisation

4.7.1 Gap Train/Test

Le Tableau 4.9 présente l'écart entre la perte d'entraînement et la perte de test pour chaque algorithme, mesuré au dernier round. Un écart faible indique une bonne généralisation, tandis qu'un écart élevé suggère un surajustement. Les algorithmes fédérés (FedAvg, FedProx) présentent systématiquement des écarts plus faibles que Local SGD, confirmant l'effet régularisateur de l'agrégation.

TABLE 4.9 – Écart entre perte d'entraînement et perte de test (round 15).

Algorithme	IID		Non-IID	
	Test Loss	Gap	Test Loss	Gap
Local SGD	0,5561	Élevé	0,2443	Modéré
FedAvg	0,3414	Faible	0,2093	Faible
FedAdam	0,4721	Modéré	0,2676	Modéré
FedProx	0,3573	Faible	0,2390	Faible

4.7.2 Effet régularisateur de l'agrégation

L'agrégation par moyenne pondérée des paramètres dans FedAvg et FedProx agit comme un mécanisme de régularisation implicite. En moyennant les paramètres de modèles entraînés sur des sous-ensembles différents, le modèle global tend à conserver les caractéristiques partagées par tous les clients (les features discriminantes universelles) tout en atténuant les caractéristiques spécifiques à un seul client (le bruit ou les biais locaux). Ce phénomène est analogue au model averaging en apprentissage d'ensemble, et explique pourquoi les modèles fédérés généralisent mieux que les modèles isolés malgré un accès indirect aux données de tous les clients.

De plus, le terme proximal de FedProx ajoute un second niveau de régularisation en pénalisant les écarts importants par rapport au modèle global pendant l'entraînement local. Cette double régularisation (agrégation + proximal) explique la convergence particulièrement lisse et la stabilité supérieure de FedProx observées dans nos expériences, même si cela peut légèrement limiter la performance maximale atteignable.

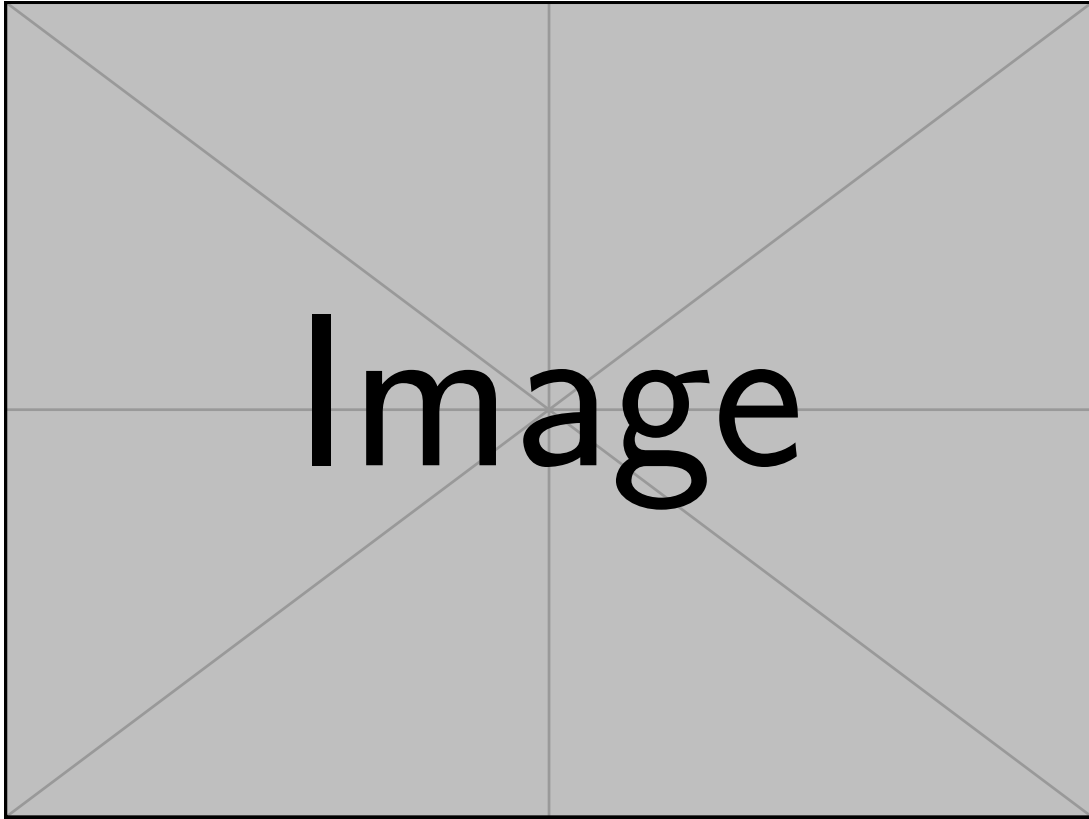


FIGURE 4.4 – Courbes de convergence de la loss au fil des rounds. Les oscillations de FedAdam en IID sont clairement visibles, tandis que FedProx maintient une décroissance monotone dans les deux scénarios.

Chapitre 5

Discussion et Perspectives

5.1 Synthèse des résultats

Le Tableau 5.1 constitue le tableau de synthèse principal de cette étude, réunissant l'ensemble des métriques pour les quatre algorithmes dans les deux scénarios de distribution. Cette vue d'ensemble permet de dégager les conclusions suivantes. FedAvg offre les meilleures performances brutes (accuracy, F1) dans les deux scénarios, confirmant son statut d'algorithme de référence pour des configurations simples avec un nombre raisonnable de clients et une hétérogénéité modérée. FedProx offre la convergence la plus stable et régulière, avec les variances de loss les plus faibles, ce qui en fait un choix recommandé pour les déploiements où la prévisibilité est critique.

FedAdam, malgré des fondements théoriques solides, sous-performe dans notre configuration. Cela s'explique par l'interaction entre l'optimiseur Adam utilisé localement et l'optimiseur Adam côté serveur, qui crée une « double adaptation » potentiellement déstabilisante. Ce résultat est cohérent avec les observations de Reddi et al. [2], qui recommandent l'utilisation de SGD côté client avec FedAdam côté serveur. Local SGD confirme sa pertinence limitée comme baseline : la collaboration fédérée apporte un gain systématique en performance et en généralisation.

TABLE 5.1 – Tableau de synthèse générale — toutes métriques.

Mode	Algo.	Acc.	F1	Loss	Comm. (Mo)	R→85%
IID	Local SGD	0,808	0,790	0,556	0	—
	FedAvg	0,885	0,884	0,341	2,43	5
	FedAdam	0,861	0,859	0,472	2,43	9
	FedProx	0,875	0,877	0,357	2,43	7
Non-IID	Local SGD	0,914	0,804	0,244	0	2
	FedAvg	0,914	0,863	0,209	2,43	2
	FedAdam	0,885	0,830	0,268	2,43	2
	FedProx	0,904	0,852	0,239	2,43	2

5.2 Recommandations par scénario

Sur la base de nos résultats expérimentaux et de l’analyse théorique, nous formulons les recommandations suivantes pour le choix de l’algorithme fédéré en fonction du contexte de déploiement.

Dans un scénario de **consortium hospitalier homogène** (hôpitaux de taille similaire, protocoles standardisés, populations comparables), **FedAvg** est le choix optimal en raison de sa simplicité d’implémentation, de ses performances élevées et de sa faible empreinte computationnelle. L’absence de terme de régularisation supplémentaire n’est pas pénalisante lorsque les données sont approximativement IID.

Dans un scénario de **consortium hétérogène** (hôpitaux spécialisés, populations distinctes, volumes de données variables), **FedProx** est recommandé pour sa capacité à maintenir une convergence stable malgré l’hétérogénéité des données. Le paramètre μ doit être ajusté en fonction du degré d’hétérogénéité : un μ plus élevé est recommandé pour des distributions très déséquilibrées.

L’utilisation de **FedAdam** est conseillée uniquement lorsque l’optimiseur local est SGD (et non Adam), pour éviter la double adaptation. Cette configuration peut être avantageuse pour des modèles de grande taille avec des échelles de gradient très hétérogènes entre les différentes couches du réseau.

5.3 Analyse critique et limites

5.3.1 Limites du dataset

Plusieurs limites liées au dataset doivent être reconnues. Premièrement, la taille de 1025 échantillons est modeste pour une partition en 8 clients, résultant en environ 100 échantillons d’entraînement par client. Cette taille réduite peut amplifier la variance des

résultats entre différentes exécutions et entre différents clients. Deuxièmement, le dataset ne contient pas de composante temporelle, ce qui empêche l'étude du temporal skew. Troisièmement, les données proviennent initialement d'une source unique et sont artificiellement partitionnées, ce qui ne capture pas la véritable hétérogénéité inter-institutionnelle en termes de protocoles de mesure, de calibration des instruments, ou de pratiques cliniques.

5.3.2 Limites de la simulation

Notre simulation adopte un mode de participation complète (full participation) où tous les clients participent à chaque round, ce qui est optimiste pour un déploiement réel où certains hôpitaux pourraient être temporairement indisponibles. L'exécution séquentielle des clients (plutôt que parallèle) élimine les problèmes de synchronisation mais ne reflète pas les délais réels de communication réseau. L'absence de simulation de latence réseau, de perte de paquets et de clients « traînants » (stragglers) constitue une simplification significative. Dans un déploiement réel, un hôpital avec un matériel moins performant ou une connexion réseau instable pourrait retarder l'ensemble du processus dans un schéma synchrone.

5.3.3 Limites de sécurité

Bien que l'apprentissage fédéré n'échange pas les données brutes, les paramètres du modèle peuvent potentiellement révéler des informations sur les données d'entraînement. Les attaques par inversion de gradient (gradient inversion attacks) peuvent, dans certaines conditions, reconstruire des échantillons d'entraînement à partir des gradients ou des mises à jour de paramètres échangés. De même, les attaques par empoisonnement du modèle (model poisoning), où un client malveillant envoie des paramètres corrompus, n'ont pas été testées dans notre cadre expérimental. Ces aspects de sécurité sont critiques pour un déploiement en environnement médical réel et constituent un axe de travail futur important.

5.4 Perspectives futures

5.4.1 Confidentialité différentielle

La confidentialité différentielle (Differential Privacy, DP) est une technique mathématiquement rigoureuse qui offre des garanties formelles de confidentialité en ajoutant un bruit calibré aux gradients ou aux paramètres avant leur envoi au serveur. Formellement, un mécanisme $\mathcal{M}$ satisfait la (ϵ, δ) -confidentialité différentielle si, pour tout sous-ensemble

S de sorties possibles et pour tous datasets adjacents $\mathcal{D}, \mathcal{D}'$ (différant d'un seul échantillon) : $\Pr[\mathcal{M}(\mathcal{D}) \in S] \leq e^\epsilon \Pr[\mathcal{M}(\mathcal{D}') \in S] + \delta$. L'intégration de DP dans notre pipeline fédéré nécessiterait un équilibrage délicat entre le niveau de confidentialité (paramètre ϵ) et la précision du modèle, car l'ajout de bruit dégrade inévitablement les performances.

5.4.2 Agrégation sécurisée

Le calcul multipartite sécurisé (Secure Multi-Party Computation, SMPC) permettrait au serveur de calculer l'agrégation des paramètres sans jamais voir les contributions individuelles des clients. Des protocoles comme le masquage additif (additive masking) ou le partage de secrets de Shamir pourraient être intégrés pour empêcher un serveur curieux d'accéder aux paramètres individuels de chaque hôpital. Cette couche de sécurité supplémentaire est essentielle pour des déploiements dans des environnements où le serveur central n'est pas pleinement de confiance, comme un consortium inter-établissements sans autorité centrale.

5.4.3 Personnalisation fédérée

Les algorithmes de personnalisation fédérée, tels que Ditto et Per-FedAvg, visent à combiner les avantages du modèle global (généralisation) et des modèles locaux (spécialisation). Ditto entraîne simultanément un modèle global et un modèle personnalisé pour chaque client, ce dernier étant régularisé par le modèle global. Per-FedAvg utilise le méta-apprentissage (MAML) pour produire un modèle global qui peut être rapidement adapté à chaque client avec quelques pas de gradient local. Ces approches sont particulièrement pertinentes dans notre contexte médical, où chaque hôpital pourrait bénéficier d'un modèle ajusté à sa population spécifique tout en exploitant les connaissances générales acquises auprès de l'ensemble du consortium.

5.4.4 Passage à l'échelle

Plusieurs axes permettraient de passer à l'échelle. La sélection partielle de clients (client sampling) réduirait le coût de communication en ne sélectionnant qu'un sous-ensemble de clients à chaque round. L'apprentissage fédéré asynchrone, où le serveur agrège les mises à jour dès leur réception sans attendre tous les clients, éliminerait le problème des stragglers. L'agrégation hiérarchique, avec des serveurs intermédiaires régionaux, pourrait réduire la charge du serveur central et améliorer la tolérance aux pannes. Enfin, des techniques de compression des modèles (quantification, élagage, distillation) permettraient de réduire davantage le volume de communication sans dégrader significativement les performances.

5.4.5 Déploiement réel

Le passage d'une simulation à un déploiement réel nécessiterait plusieurs composants supplémentaires. Au niveau de la sécurité réseau, l'utilisation de TLS (Transport Layer Security) pour le chiffrement des communications et l'authentification mutuelle (mutual TLS) pour vérifier l'identité des clients seraient indispensables. L'intégration dans l'infrastructure IT hospitalière nécessiterait une conformité avec les normes de sécurité en vigueur et une intégration avec les systèmes d'information hospitaliers existants. Le déploiement sur des dispositifs edge computing à proximité des serveurs hospitaliers réduirait la latence et améliorerait la fiabilité. Enfin, un système de monitoring continu devrait surveiller la qualité du modèle, détecter les dérives de distribution et alerter en cas d'anomalie.

Chapitre 6

Conclusion Générale

Ce projet a permis de mener une étude comparative approfondie de quatre algorithmes d’optimisation fédérée dans le contexte de la détection de maladies cardiaques. À travers la simulation d’un consortium de huit hôpitaux exploitant le dataset Heart Disease UCI, nous avons évalué les performances, la convergence, la stabilité et l’efficacité de Local SGD, FedAvg, FedAdam et FedProx dans des conditions IID et Non-IID. Les conclusions principales sont les suivantes.

1. La collaboration fédérée surpasse l’apprentissage isolé. Les résultats démontrent de manière univoque que les algorithmes fédérés (FedAvg, FedAdam, FedProx) surpassent systématiquement Local SGD en termes de score F1 et de capacité de généralisation. En scénario IID, FedAvg atteint un F1 de 0,884 contre 0,790 pour Local SGD, soit un gain de 9,4 points. En Non-IID, FedAvg atteint 0,863 contre 0,804 pour Local SGD. L’agrégation des paramètres agit comme un mécanisme de régularisation implicite qui réduit le surajustement et améliore la généralisation.

2. FedProx offre la convergence la plus stable. Le terme de régularisation proximale ($\mu = 0.1$) de FedProx réduit efficacement la dérive des clients et produit des courbes de convergence monotonement décroissantes. Cette propriété est particulièrement précieuse en contexte médical, où la prévisibilité du comportement du modèle est essentielle pour obtenir la confiance des praticiens.

3. Le coût de communication est négligeable pour les modèles tabulaires. Avec seulement 2,43 Mo échangés sur 15 rounds pour un modèle de 2 657 paramètres, l’apprentissage fédéré est parfaitement viable même sur des connexions réseau modestes. Ce résultat plaide en faveur de l’utilisation de modèles adaptés à la nature des données plutôt que de modèles sur-dimensionnés.

4. L’hétérogénéité modérée n’est pas rédhibitoire. Contrairement aux attentes théoriques, le scénario Non-IID ($\alpha = 0.5$) ne dégrade pas systématiquement les performances dans notre configuration. Ce résultat encourage le déploiement de l’apprentissage fédéré dans des environnements hospitaliers réels où une certaine hétérogénéité est inévitable.

Ce travail constitue une contribution concrète à l'étude de l'apprentissage fédéré appliqué au domaine médical, en fournissant une implémentation complète et une analyse comparative rigoureuse de quatre algorithmes fondamentaux. La simulation développée, bien que simplifiée par rapport à un déploiement réel, démontre la faisabilité et les bénéfices du paradigme fédéré pour la détection de maladies cardiaques. Les perspectives ouvertes — confidentialité différentielle, agrégation sécurisée, personnalisation, passage à l'échelle — dessinent un programme de recherche prometteur pour faire de l'apprentissage fédéré un outil clinique de routine au service de la santé publique.

Bibliographie

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.
- [2] S. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, “Adaptive federated optimization,” in *Proc. 9th Int. Conf. Learning Representations (ICLR)*, 2021.
- [3] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proc. Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450.
- [4] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated learning : Challenges, methods, and future directions,” *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.
- [5] P. Kairouz, H. B. McMahan, B. Avent, *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [6] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, “Federated learning with non-IID data,” *arXiv preprint arXiv :1806.00582*, 2018.
- [7] K. Bonawitz, H. Eichner, W. Grieskamp, *et al.*, “Towards federated learning at scale : A system design,” in *Proc. Machine Learning and Systems (MLSys)*, vol. 1, 2019, pp. 374–388.
- [8] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, “Heart disease dataset,” UCI Machine Learning Repository, 1988. <https://archive.ics.uci.edu/ml/datasets/heart+disease>

Annexe A

Code source principal

Cette annexe présente les extraits de code les plus significatifs de l'implémentation. Le code complet est disponible dans le dépôt du projet.

A.1 Fonction d'agrégation

La fonction `aggregate()` implémente la moyenne pondérée des paramètres (FedAvg) :

```
1 def aggregate(weights_results):
2     """Weighted average of model weights (FedAvg aggregation)."""
3     total_samples = sum(n for w, n in weights_results)
4     agg_weights = []
5     for i, (weights, n) in enumerate(weights_results):
6         frac = n / total_samples
7         if i == 0:
8             agg_weights = [w * frac for w in weights]
9         else:
10            agg_weights = [aw + w * frac
11                           for aw, w in zip(agg_weights, weights)]
12     return agg_weights
```

Listing A.1 – Fonction d'agrégation FedAvg.

A.2 Entraînement local du client

La méthode `LocalClient.train()` gère l'entraînement local avec le support du terme proximal pour FedProx :

```
1 def train(self, global_model, epochs, lr,
2           strategy="FedAvg", mu=0.1):
3     model = copy.deepcopy(global_model)
```

```

4     model.to(self.device)
5     model.train()
6     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
7     criterion = nn.BCEWithLogitsLoss()
8     global_params = [p.clone().detach()
9                       for p in global_model.parameters()]
10
11    epoch_loss, n_batches = 0.0, 0
12    for epoch in range(epochs):
13        for inputs, labels in self.trainloader:
14            inputs = inputs.to(self.device)
15            labels = labels.to(self.device).float().unsqueeze(1)
16            optimizer.zero_grad()
17            outputs = model(inputs)
18            loss = criterion(outputs, labels)
19
20            if strategy == "FedProx":
21                proximal_term = 0.0
22                for param, gp in zip(model.parameters(),
23                                     global_params):
24                    proximal_term += ((param - gp)**2).sum()
25                loss += (mu / 2.0) * proximal_term
26
27            loss.backward()
28            optimizer.step()
29            epoch_loss += loss.item()
30            n_batches += 1
31
32    avg_train_loss = epoch_loss / max(n_batches, 1)
33    updated_weights = [p.cpu().detach().numpy()
34                      for p in model.parameters()]
35    return updated_weights, self.num_train, avg_train_loss

```

Listing A.2 – Méthode d’entraînement local avec support FedProx.

A.3 Boucle principale de simulation

La fonction `run_simulation()` orchestre le processus fédéré complet, incluant le support des trois stratégies (FedAvg, FedAdam, FedProx) :

```

1 for r in range(1, num_rounds + 1):
2     round_start = time.perf_counter()

```

```

3 weights_results = []
4 for client in clients:
5     weights, n, train_loss = client.train(
6         global_model, epochs=epochs, lr=0.001,
7         strategy=strategy_name
8     )
9     weights_results.append((weights, n))
10
11 agg_weights = aggregate(weights_results)
12 divergence = compute_divergence(
13     [w for w, n in weights_results], global_model
14 )
15
16 if strategy_name == "FedAdam":
17     global_weights = [p.detach().numpy()
18                       for p in global_model.parameters()]
19     delta_t = [gw - aw
20                for gw, aw in zip(global_weights, agg_weights)]
21     for i in range(len(m_t)):
22         m_t[i] = beta1*m_t[i] + (1-beta1)*delta_t[i]
23         v_t[i] = beta2*v_t[i] + (1-beta2)*(delta_t[i]**2)
24         global_weights[i] = (global_weights[i]
25                               - eta * m_t[i] / (np.sqrt(v_t[i]) + tau))
26     new_params = global_weights
27 else:
28     new_params = agg_weights
29
30 # Update global model
31 params_dict = zip(global_model.state_dict().keys(),
32                   new_params)
33 state_dict = OrderedDict(
34     {k: torch.tensor(v) for k, v in params_dict})
35 global_model.load_state_dict(state_dict, strict=True)

```

Listing A.3 – Boucle principale de simulation fédérée (extrait).

Annexe B

Glossaire des termes techniques

Apprentissage fédéré (Federated Learning) Paradigme d'apprentissage automatique distribué permettant l'entraînement collaboratif d'un modèle sans centralisation des données.

Agrégation Opération côté serveur combinant les paramètres des modèles locaux en un modèle global. La méthode la plus courante est la moyenne pondérée par le nombre d'échantillons.

BCEWithLogitsLoss Fonction de perte combinant une couche sigmoïde et l'entropie croisée binaire, utilisée pour la classification binaire avec une stabilité numérique améliorée.

Client drift (dérive des clients) Phénomène par lequel les modèles locaux s'éloignent du modèle global après un entraînement local sur des données hétérogènes.

Confidentialité différentielle (Differential Privacy) Mécanisme mathématique ajoutant un bruit calibré pour garantir formellement qu'aucun échantillon individuel ne peut être identifié à partir des sorties du modèle.

Distribution de Dirichlet Distribution de probabilité sur le simplexe, utilisée dans la littérature FL pour simuler des partitions Non-IID contrôlées.

Dropout Technique de régularisation désactivant aléatoirement une fraction p des neurones pendant l'entraînement pour prévenir le surajustement.

FedAvg (Federated Averaging) Algorithme fédéré de référence combinant SGD local et agrégation par moyenne pondérée.

FedAdam Variante adaptative de FedAvg utilisant les mécanismes de momentum et de variance d'Adam côté serveur pour les mises à jour du modèle global.

FedProx Extension de FedAvg ajoutant un terme de régularisation proximale $\frac{\mu}{2}\|w - w^{(t)}\|^2$ à l'objectif local pour limiter la dérive des clients.

IID (Independent and Identically Distributed) Hypothèse statistique selon laquelle les données de chaque client suivent la même distribution que la population

globale.

Label skew Type d'hétérogénéité statistique où la distribution des classes diffère entre les clients.

MLP (Multilayer Perceptron) Réseau de neurones complètement connecté composé de couches linéaires et de fonctions d'activation non linéaires.

Non-IID (Non-Independent and Identically Distributed) Situation où les distributions de données diffèrent entre les clients, constituant le défi central de l'apprentissage fédéré.

Pseudo-gradient Dans FedAdam, différence entre le modèle global courant et le résultat de l'agrégation, traitée comme un gradient pour l'optimisation côté serveur.

ReLU (Rectified Linear Unit) Fonction d'activation définie par $\max(0, x)$, largement utilisée dans les réseaux de neurones modernes pour sa simplicité et ses propriétés de gradient favorable.

Round de communication Cycle complet comprenant la distribution du modèle global, l'entraînement local sur tous les clients, et l'agrégation des mises à jour.

SGD (Stochastic Gradient Descent) Algorithme d'optimisation itératif utilisant des estimations stochastiques du gradient sur des mini-batches.

Terme proximal Dans FedProx, pénalité quadratique $\frac{\mu}{2} \|w - w^{(t)}\|^2$ ajoutée à la perte locale pour contraindre les mises à jour à rester proches du modèle global.

Z-score (standardisation) Transformation affine $z = (x - \mu) / \sigma$ centrant les données autour de zéro avec une variance unitaire.